

ENLANG FOR DEVELOPERS

The Definitive Guide to Building Full-Stack Software with Natural English

Author: Spandan Prayas Patra

Version: 2.0.0 — Enterprise Specification Edition

Publisher: Universal EnLang Foundation Press

"Programming should be as clear, expressive, and frictionless as human thought. EnLang makes natural English a first-class, production-grade programming language across the entire software stack."

— Spandan Prayas Patra

TABLE OF CONTENTS

- Chapter 1: Introduction & Philosophy
 - Chapter 2: Variables, Types & Expressions
 - Chapter 3: Control Flow
 - Chapter 4: Functions & Classes
 - Chapter 5: Pattern Matching
 - Chapter 6: Exception Handling
 - Chapter 7: Collections — Lists, Arrays, Dictionaries, Sets
 - Chapter 8: String & Math Operations
 - Chapter 9: File I/O & System Operations
 - Chapter 10: Frontend Markup — `.enlgf` to HTML5
 - Chapter 11: Styling & Design Systems — `.enlgd` to CSS3
 - Chapter 12: Client Scripts — `.enlgs` to JavaScript ES6+
 - Chapter 13: Database Schemas & Queries — `.enlgdb` to SQL
 - Chapter 14: Native NLP & AI Primitives
 - Chapter 15: EnLang Web Server & App Assembly
 - Chapter 16: Complete Real-World Case Study — Lumina Workspace
 - Appendix A: Complete Syntax Reference
 - Appendix B: CLI & Package Manager (EPM) Reference
-

CHAPTER 1: INTRODUCTION & PHILOSOPHY

1.1 The Vision

EnLang was designed by Spandan Prayas Patra to invert the traditional programming paradigm. Instead of forcing developers to adapt to cryptic syntax rules, EnLang allows expression of software logic in pure, readable, natural English — with guaranteed deterministic compilation into battle-tested native targets.

EnLang is not a pseudo-code generator or an LLM wrapper. It is a deterministic, multi-target compiler and runtime engine.

1.2 Domain Architecture

.enlg	Backend Logic	→ Python 3
.enlgf	Web Structure	→ HTML5
.enlgd	Styling & Design	→ CSS3
.enlgs	Client Scripts	→ JavaScript ES6+
.enlgdb	Database & Schema	→ SQL (SQLite)

1.3 The EnLang Pipeline

```
graph TD
    A[EnLang Source] --> B[Grammar.py (Pattern Matching Engine)]
    B --> C[Transpiler.py (1:1 Target Code Generator)]
    C --> D[Interpreter.py (Python exec() Runtime)]
    D --> E[Native Output (HTML / CSS / JS / SQL / Python)]
```

1.4 Design Principles

- ZERO hardcoded class names or style injections
- Natural English flexibility — multiple syntaxes for same action are all valid
- Raw native code (HTML/CSS/JS/SQL) always passes through verbatim
- Universal fallbacks handle unknown syntax gracefully
- Type annotations are optional — the language is dynamically typed by default

CHAPTER 2: VARIABLES, TYPES & EXPRESSIONS

2.1 Flexible Variable Assignment

EnLang supports multiple natural syntaxes for the same operation. All are valid:

```
set score to 100
let score = 100
store 100 in score
score is 100
```

2.2 Optional Type Declarations (v2.0)

Type annotations are completely optional. They help document intent but do not restrict dynamic usage.

Primitive Types

```
define number age as 20
define decimal price as 99.50
define text username as "Spandan"
define boolean isActive as true
```

Collection Types

```
define list fruits as ["Apple", "Banana", "Cherry"]
define array tags as ["ai", "nlp", "compiler"]
define dictionary profile as {"name": "Spandan", "role": "Architect"}
define set unique_ids as {101, 102, 103}
```

Declare empty typed variables (with safe default values)

```
define list items          # items = []
define dictionary config   # config = {}
define set visited         # visited = set()
define number counter      # counter = 0
define text message       # message = ""
define boolean flag        # flag = False
```

2.3 Natural English Operators

Natural English	Python Equivalent
is equal to	==
is not equal to / is not	!=

is greater than	>
is less than	<
is greater than or equal to	>=
is less than or equal to	<=
plus	+
minus	-
times	*
divided by	/
modulo	%
power of	**
true / false	True / False
null / none	None

Examples

```
set total to price plus tax
set discount to price times 0.2
set result to value power of 2
set remainder to total modulo 7
```

```
if score is greater than or equal to 90 then:
    display "Excellent!"
```

2.4 Type Conversion

```
set raw to "42"
convert raw to int and store in count
display count

convert count to str and store in label
convert price to int and store in whole_price
convert flag to str and store in flag_text
```

Supported type names: `int`, `integer`, `float`, `double`, `str`, `string`, `bool`, `boolean`, `list`, `tuple`, `set`

CHAPTER 3: CONTROL FLOW

3.1 If / Otherwise If / Else

```
set score to 85

if score is greater than 90 then:
  display "Grade: A+"
otherwise if score is greater than 80 then:
  display "Grade: A"
otherwise if score is greater than 70 then:
  display "Grade: B"
else:
  display "Grade: C"
```

3.2 Loops

Repeat N Times

```
repeat 5 times do:
  display "Batch item processed"
```

For Each Loop

```
create list users with items "Alice", "Bob", "Charlie"

for each user in users do:
  display "Processing user: " plus user
```

While Loop

```
set count to 1

while count is less than 10 do:
  display count
  increment count by 1
```

Loop Control

```
for each item in catalog do:
  if item is equal to "STOP":
    break
  display item

for each num in numbers do:
  if num modulo 2 is equal to 0:
    continue
  display "Odd: " plus num
```

CHAPTER 4: FUNCTIONS & CLASSES

4.1 Functions

```
function greet(name):
    display "Hello, " plus name

function calculate_tax(price, rate):
    set tax to price times rate
    return tax

set amount to calculate_tax(1000, 0.18)
display "Tax: " plus amount
```

4.2 Classes (v2.0)

```
class User:
    property name
    property age

    function greet():
        display "Hello, I am " plus self.name

u = User()
u.name = "Spandan"
u.greet()
```

4.3 Class Inheritance (v2.0)

```
class Animal:
    function speak():
        display "..."

class Dog extends Animal:
    function speak():
        display "Woof!"

class Cat extends Animal:
    function speak():
        display "Meow!"

d = Dog()
d.speak()
```

4.4 OOP with Constructor (Raw Python passthrough)

Since class constructors use `__init__`, raw Python inside `.enlg` passes through cleanly:

```
class Product:
    def __init__(self, name, price):
        self.name = name
        self.price = price

    function get_info():
        return self.name plus " - Rs." plus str(self.price)

p = Product("Laptop", 75000)
```

```
display p.get_info()
```

CHAPTER 5: PATTERN MATCHING (v2.0)

Pattern matching is EnLang's clean, safe alternative to [switch/case](#). There is no fallthrough bug risk — every [case](#) block is automatically isolated.

5.1 Basic Match

```
set role to "admin"

match role:
  case "admin":
    display "Full Management Access"
  case "editor":
    display "Content Editing Access"
  case "viewer":
    display "Read-Only Access"
  default:
    display "Guest Access"
end match
```

5.2 Match with Multiple Values per Case

```
set grade to 85

match grade:
  case 90, 95, 100:
    display "Gold Medal"
  case 80, 85, 88:
    display "Silver Medal"
  case 70, 75, 78:
    display "Bronze Medal"
  default:
    display "Keep Improving"
end match
```

5.3 Match with Conditional Case

```
set score to 72

match score:
  case is greater than 90:
    display "Exceptional"
  case is greater than 75:
    display "Good"
  case is greater than 50:
    display "Average"
  default:
    display "Below Average"
end match
```

CHAPTER 6: EXCEPTION HANDLING (v2.0)

6.1 Try / Except / Finally

```
try:
  fetch url "https://api.lumina.dev/data" and store in response
  display response
except:
  display "Network error – could not reach server"
finally:
  display "Request attempt complete"
```

6.2 Typed Exception Handling (Raw Python syntax)

```
try:
  set result to 100 divided by 0
except ZeroDivisionError:
  display "Cannot divide by zero"
except Exception:
  display "Unknown error occurred"
finally:
  display "Done"
```

6.3 Raising Exceptions (v2.0)

```
raise ValueError with message "Score cannot be negative"

raise TypeError with message "Expected a number, got text"
```


6.4 Throw Error (Shorthand)

```
throw error "Authentication failed – invalid token"

throw exception "Database connection timeout"
```

CHAPTER 7: COLLECTIONS

7.1 Lists

```
create list fruits with items "Apple", "Banana", "Cherry"

add "Mango" to fruits

get item at index 0 from fruits and store in first

remove item at index 2 from fruits

get length of fruits and store in total

sort fruits

for each fruit in fruits do:
  display fruit

join fruits with ", " and store in label
```

7.2 Arrays (Alias of List)

```
define array scores as [85, 92, 78, 95, 60]

add 88 to scores

get length of scores and store in count
display "Total scores: " plus count
```

7.3 Dictionaries / Maps

```
create map config
set key "host" in map config to "localhost"
set key "port" in map config to 8080
set key "debug" in map config to true

get key "host" from map config and store in server_host
display "Server: " plus server_host
```

7.4 Sets

```
define set visited_pages

# Use raw Python for set operations
visited_pages.add("/home")
visited_pages.add("/about")
visited_pages.add("/home") # Duplicate ignored

display str(len(visited_pages)) # Output: 2
```

7.5 Checking Membership

```
check if "Mango" is in list fruits and store in found

if found is equal to True then:
    display "Mango found!"
```

CHAPTER 8: STRING & MATH OPERATIONS

8.1 String Operations

```
set name to "spandan"

convert name to uppercase and store in upper_name
display upper_name # SPANDAN

convert name to lowercase and store in lower_name
display lower_name # spandan

trim " hello world " and store in clean
display clean # hello world

split "a,b,c" by "," and store in parts
display str(parts) # ['a', 'b', 'c']

replace "world" with "EnLang" in "hello world" and store in result
display result # hello EnLang

check if name contains "span" and store in has_span
display str(has_span) # True
```

8.2 Math Operations

```
set x to 3.14159
round x to 2 decimal places and store in r
display r # 3.14

get absolute value of -42 and store in abs_val
display abs_val # 42

define list nums as [3, 1, 9, 5, 7]
get minimum of nums and store in lo
get maximum of nums and store in hi
get sum of nums and store in total

display lo # 1
display hi # 9
display total # 25
```

8.3 DateTime

```
get current time and store in now
display str(now) # 2026-07-24 17:53:00.138907

get current date and store in today
display str(today) # 2026-07-24

get current timestamp and store in ts
display ts # Unix epoch integer

format date now as "%d %B %Y" and store in pretty
display pretty # 24 July 2026
```

8.4 Sleep

```
display "Starting process..."
sleep 2 seconds
display "Done after 2 second delay"

sleep 500 ms
```

CHAPTER 9: FILE I/O & SYSTEM OPERATIONS

9.1 File Read & Write

```
write "Session started: 2026-07-24" to file "logs.txt"

read file "logs.txt" into log_content
display log_content
```

9.2 Environment Variables

```
get environment variable "DATABASE_URL" and store in db_url
display db_url

set environment variable "APP_MODE" to "production"
```

9.3 File System

```
check if file "config.json" exists and store in config_exists

if config_exists is equal to True then:
    display "Config found"

create directory "output/reports"

list files in "output" and store in files
display str(files)
```

9.4 Shell Commands

```
run command "git status"

run command "pip install requests"
```

9.5 Hashing & Security

```
hash "spandan123" with sha256 and store in hashed_pw
display hashed_pw
```

9.6 HTTP Fetch

```
fetch url "https://api.github.com/users/octocat" and store in data
display data
```

9.7 Module Imports

```
import module math
import module json
import module datetime as dt
import module os

display str(math.pi)
```

CHAPTER 10: FRONTEND MARKUP —

`.enlgf` → HTML5

The `.enlgf` domain generates clean, semantic HTML5 with zero hardcoded class injections.

10.1 Page Head Elements

```
page title "AURA Workspace"
page meta description content "The intelligent developer workspace"
page charset UTF-8
```

10.2 Universal Tag Engine

Any valid HTML5 tag works:

```
create header with class "site-header"
  create nav with class "navbar"
    create h1 with text "AURA"
    create a with id "logo-link" with href "#home" with text "Home"
  close nav
close header

create main with class "container"
  create section named hero with class "hero-block"
    create h2 with text "Welcome Back, Developer"
    create p with text "Your environment is ready."
  close section
close main
```

10.3 High-Level Component Sugar

```
# Composite Hero Section
create hero with title "Lumina Workspace", subtitle "Built with EnLang"

# Navigation with links
create nav with links "Home", "Features", "Docs", "Contact"

# Smart Form
create form named loginForm with fields Email, Password and action "/api/auth"

# Data Card
create card named statsCard with title "Active Users", description "12,840 online now"

# Data Table
create table with headers "ID", "User", "Role", "Status", "Last Seen"
```

10.4 Media & Void Elements

```
create image named logo with src "/assets/logo.png", alt "AURA Logo"
create image with src "/assets/hero.webp", alt "Hero Banner"
linebreak
horizontal rule
create input with type "email", placeholder "Enter your email", name "email"
```

10.5 SVG & Vector Graphics

```
create svg with width 200, height 200, viewBox "0 0 200 200"
  create circle with cx 100, cy 100, r 80, fill "#6366f1", stroke "#fff", stroke width 3
  create rect with x 40, y 140, width 120, height 20, fill "#38bdf8"
  create path with d "M10 80 Q 95 10 180 80", fill "none", stroke "#f59e0b"
close svg
```

10.6 External Scripts & Stylesheets

```
create stylesheet with href "https://fonts.googleapis.com/css2?family=Inter"
create script with src "/js/app.js"
```

10.7 Raw HTML Passthrough

Any raw HTML tag written directly in `.enlgf` passes through verbatim:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
</head>
```

CHAPTER 11: STYLING & DESIGN —

`.enlgd` → CSS3

The `.enlgd` domain produces clean CSS3 with zero auto-injected framework styles.

11.1 Theme & Custom Properties

```
define theme with primary "#6366f1", background "#090d16", accent "#38bdf8"

import font "https://fonts.googleapis.com/css2?family=Inter:wght@300;400;600;700"
```